

Permission Misuse in Android Applications: Security Risks

Miguel Pombeiro and Miguel Rocha

Universidade de Évora, Évora, Portugal
{157829,158501}@alunos.uevora.pt

Abstract. The extensive use of mobile devices has made Android security a critical concern, particularly due to the large amount of sensitive data stored on smartphones. This paper begins with a brief analysis of the Android permission model, focusing on its architecture and protection levels. We then address permission misuse, a persistent issue in Android applications, highlighting the presence of over-privileged apps and the challenges developers face when mapping APIs to permissions. Additionally, we examine the role of third-party libraries, which can exploit already granted permissions through techniques such as permission piggybacking, often without user awareness. Finally, we discuss the main security and privacy risks associated with these issues, including attacks such as component hijacking and UI overlay attacks, as well as other threats targeting the permission system. We also outline some security mechanisms implemented by Google over the different Android versions.

Keywords: Android Applications · Android Security · Permission Model · Permission Misuse · Over-privileged Applications · Privacy · Security

1 Introduction

The extensive adoption of mobile devices has deeply integrated them into our daily lives which makes the security of mobile applications a critical need for our society. Android currently dominates the mobile operating system landscape with a market share of over 68%¹, powering billions of devices globally [1]. Since smartphones serve as central hubs for sensitive activities - from online banking and healthcare to social networking - they end up storing a vast amount of Personally Identifiable Information (PII) [9]. To protect this sensitive data from unauthorized access, Android relies on a robust permission-based security model that acts as the primary gatekeeper for user privacy and device integrity. Understanding and addressing vulnerabilities within this mechanism is highly relevant because malicious exploitation and improper management of these permissions introduces severe security and privacy risks [1].

At its core, the Android permission system regulates an application's access to restricted data (i.e. contacts, messages, location) and restricted hardware actions (i.e. camera, microphone). Permissions are officially categorized by Google

¹ <https://gs.statcounter.com/os-market-share/mobile/worldwide>

into different protection levels, primarily Normal, Signature, Dangerous as it will be explained in section 2 [8]. A major highlight in Android’s privacy evolution was the shift from an "all-or-nothing" install time model to a dynamic runtime permission model introduced in Android 6.0 (Marshmallow). This shift empowered users to selectively grant or revoke permissions at their discretion [13]. At its core, the entire framework operates on the Principle of Least Privilege (PoLP), which dictates that an application should only request the absolute minimum permissions necessary to execute their core functionalities [1] [9].

Despite extensive documentation and evolving safeguards, the Android ecosystem is increasingly burdened by permission misuse and the proliferation of over-privileged applications [1] [11]. This means that many applications request permissions far beyond their functional requirements. This over-privilege frequently originates from poor development practices - where developers create wrong permission mappings for their needs or misconfigure custom permissions [11] [14]. On the opposite side, malicious actors intentionally exploit the permission system to make attacks such as ransomware or unauthorized interception of private data [2].

2 Android Permission Model

The Android Permission Model is the basis for the platform’s multilayered defense strategy. It is designed to strike a balance between providing rich applications functionality and protecting the user’s privacy [6]. Because smartphones handle highly sensitive data and hardware operations, the Android system cannot inherently trust third-party applications. Instead, it employs a multiparty authorization model, where access to protected resources requires explicit consent [6]. This section analyzes the security architecture that imposes these boundaries and the categorization of permissions into distinct protection levels.

2.1 The Security Architecture

Android’s security architecture is built upon a layered software stack, relying on the Linux kernel at its foundation to implement process isolation and hardware abstraction [8]. The core of Android’s access control is the Application Sandbox [6].

When an application is installed, the Linux kernel assigns it a unique User ID (UID) [2]. Because each application runs as a separate user in its own Android Runtime (ART) virtual machine, it is completely isolated from other applications and system processes [2]. With this sandboxing it is ensured that no application can perform actions that adversely impact other apps, the operating system, or the user [2].

To manage access across these isolated sandboxes, Android utilizes three different access control mechanisms:

- **Discretionary Access Control (DAC):** Uses traditional UNIX-style permissions. The apps own their resources and have discretion to grant/deny

access to them such as changing file permissions to allow world-read access [6].

- **Mandatory Access Control (MAC):** Implemented using Security Enhanced Linux (SELinux), this policy dictates what actions are allowed. It operates on a "default deny" principle, meaning no action is allowed unless explicitly granted by the system [4] [6].
- **Android Permissions:** While DAC and MAC operate at the kernel level, Android adds a higher level of semantic permissions (e.g. camera, location, contacts). These permissions are declared on the app's manifest and enforced in the userspace by specific data/service providers [6].

2.2 Protection Levels

To implement the PoLP, Android classifies permissions in different protection levels based on the severity of the risk they pose to the system and the user's privacy [6] [14]. The developers must explicitly declare all the permissions an app needs in the `AndroidManifest.xml` file [8]. The main protection levels are:

- **Normal Permissions:** Present low risk, such as accessing the internet or vibrating the device. The system automatically grants these at install time without prompting the user [1] [2].
- **Dangerous Permissions:** Guard highly sensitive user data (location, contacts) or critical hardware (camera, microphone). Since Android 6.0, developers must write specific code to request these permissions dynamically at runtime. This model gives the users fine control, allowing them to approve, deny or revoke access at their discretion through system settings [1] [2] [4].
- **Signature Permissions:** These are granted automatically at install time, but only if the requesting app is signed with the exact same developer certificate as the app that defined the permission. This is a secure choice for developers that want to share resources between their own suite of apps without exposing it to third-parties [1] [2] [6].

3 Permission Misuse in Android Applications

Despite the improvements in the Android security architecture, permission misuse is still one of the most persistent vulnerabilities in mobile ecosystems. The complexity of managing runtime permissions often leads to implementation mistakes. This section explores how these mistakes manifest as over-privilege and how third-party libraries can exploit these vulnerabilities.

3.1 Over-privileged Applications and Misuse of Permissions

Recent studies have been showing that a massive portion of Android apps violate the PoLP, resulting in over-privileged applications that request access to data and hardware far beyond their actual needs [10].

The root cause of this over-privilege is rarely malicious intent from the developer; it is usually related to the sheer difficulty of understanding Android's API-to-permission mappings. Because the official documentation often lacks precise, up-to-date mappings for every API call, developers struggle to know exactly which permissions are required [13]. Also, since these mappings frequently change with Android versions, and to prevent applications from crashing at runtime due to a missing permission, confused developers often adopt the bad practice of over-declaring permissions "just-in-case" [11] [13].

This over-privilege expands the attack surface of the applications. Unused but granted permissions can be exploited through vulnerabilities, allowing malicious apps to perform permission escalation or ransomware through Inter-Process Communication [2]. Interestingly, a study analyzing apps from 2019 to 2023 show a shifting in dynamic in developer strategies: while benign applications are requesting increasingly larger permission sets to support complex features, malware is minimizing its permission footprint to evade the detection systems [1].

3.2 Third-Party Libraries and Hidden Permission Abuse

The modern app development process relies heavily on integrating third-party libraries (SDKs) to handle various tasks like analytics or crash reporting. From a system perspective, the SDKs are run within the exact same application sandbox and OS process. Consequently, the permission framework cannot distinguish between the main app and its libraries, meaning libraries inherit the permissions granted to the host app [5] [6].

This design trait has given rise to a big security threat known as Permission Piggybacking (or opportunistic permission usage) [5]. In this scenario third-party tracking and advertising libraries do not declare or request permissions themselves, but instead they check the host app at runtime to see which dangerous permissions have already been granted [5]. If a highly privileged app has access to the phone state or the user's location, the library will "piggyback" on those permissions and secretly harvest PII and unique device identifiers.

A recent study showcased that about 50% of embedded libraries in top Android apps exhibited permission piggybacking, a trend that remained unchanged over the last decade [5].

Because the user only sees the permission request coming from the host app, they remain completely unaware that a third party is silently abusing that trust.

4 Security and Privacy Risks

The improper management and misuse of permissions can lead to severe security and privacy risks. When ignoring the Principle of Least Privilege (PoLP), over-privileged applications expand the attack surface by requesting unwanted permissions, leaving sensitive user data and system resources vulnerable [9].

4.1 Attacks Exploiting Permissions

Malicious actors are actively designing attacks to bypass or abuse the Android permission model. As discussed in subsection 3.2, one prevalent attack technique is the permission piggybacking, where third-party libraries exploit dangerous permissions to secretly collect data [5].

Attackers can also exploit development flaws to execute component hijacking. This happens when developers fail to protect the application's exported components or misconfigure custom permissions, allowing attackers to hijack intents or escalate privileges without user interaction [7].

Another major threat is the UI Overlay attack. Malicious apps abuse permissions like `SYSTEM_ALERT_WINDOW` or accessibility services to manipulate the UI feedback loop [3].

There are many other attacks that can be performed by exploiting the permission model, such as ransomware [2], storage downgrade attack [12], among many others.

5 Android Security Mechanisms and Improvements

To effectively fight against threats of permission misuse, data leakage and malware exploitation, relying solely on developer adherence to secure coding practices is insufficient. As a result, the Android Operating System employs a continuous defense strategy, proactively restructuring its core authorization and access control frameworks over successive releases [4].

These continuous evolutions on the permission model are made to empower users and reduce the attack surface. One of the most significant changes was on Android 6.0, when permissions shifted from the risky "all-or nothing" install-time approach to dynamic runtime permissions [1].

On Android 10 and 11, multi-state, context-aware permissions were introduced [12]. Instead of a simple allow/deny choice, users can now grant access to highly sensitive resources (i.e. location, camera, etc.) "only while using the app" or on a "one-time" basis [12]. Android 11 also introduced permission auto-reset, which revokes sensitive permissions from applications that have not been used for some time, protecting users that forget about older apps [9].

Android has also tightened data access through Scoped Storage, which protects the privacy of user and app data by limiting access to specific media files rather than granting broad access to the entire shared storage directory [9]. The ecosystem also relies on Google Play Protect, a threat detection service that scans all applications from Google Play to detect and remove those exhibiting harmful behaviors [4].

6 Conclusion

The Android permission system plays an important role in the user privacy and data, but it is not as secure in practice as it looks on paper. Even though

Android security architecture has improved a lot over the years, especially with the introduction of runtime permissions, there are still many problems related to how permissions are used.

From our analysis, the majority of permission-based vulnerabilities don't come from the system itself, but rather from poor development practices. Over-privileged apps, unclear permission mappings, and the heavy use of third-party libraries all make it easier for permissions to be misused. On top of that, attackers are constantly finding new ways to take advantage of these weaknesses.

In the end, improving mobile security is not just about adding more mechanisms, but about using the existing ones correctly. Following basic principles like the Principle of Least Privilege and being more careful with permissions and external libraries can already make a big difference. At the same time, better tools and clearer documentation would help developers avoid common mistakes.

Overall, permission misuse remains a prevalent and real threat in Android applications, and fixing it requires effort from both developers and the platform itself.

Declaration of Generative AI in the Writing Process

The authors used AI tools such as Gemini, ChatGPT and Claude to assist with summarizing, organizing and condensing source material. These tools were also used to proofread and spell check. The content was reviewed and written only by the authors.

References

1. Alkinoon, A., Dang, T.C., Alghuried, A., Alghamdi, A., Choi, S., Mohaisen, M., Wang, A., Salem, S., Mohaisen, D.: A comprehensive analysis of evolving permission usage in android apps: Trends, threats, and ecosystem insights. *Journal of Cybersecurity and Privacy* **5**(3) (2025). <https://doi.org/10.3390/jcp5030058>, <https://www.mdpi.com/2624-800X/5/3/58>
2. Almomani, I.M., Khayer, A.A.: A comprehensive analysis of the android permissions system. *IEEE Access* **8**, 216671–216688 (2020). <https://doi.org/10.1109/ACCESS.2020.3041432>
3. Choe, Y., Yu, H., Kim, T., Lee, S., Lee, H., Kim, H.: (in)visible privacy indicator: Security analysis of privacy indicator on android devices. In: *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. p. 1630–1643. ASIA CCS '24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3634737.3645014>
4. Google: Android security paper 2024. Tech. rep., Google (2024)
5. Heid, K., Sonntag, E.J., Heider, J.: Revisiting permission piggybacking of third-party libraries in android apps. In: *ICISSP* (1). pp. 39–46 (2025)
6. Mayrhofer, R., Stoep, J.V., Brubaker, C., Kravovich, N.: The android platform security model (2023). *ACM Transactions on Privacy and Security* (2023), <http://arxiv.org/abs/1904.05572>

7. Mazuera-Rozo, A., Escobar-Velásquez, C., Espitia-Acero, J., Vega-Guzmán, D., Trubiani, C., Linares-Vásquez, M., Bavota, G.: Taxonomy of security weaknesses in java and kotlin android apps. *J. Syst. Softw.* **187**(C) (May 2022). <https://doi.org/10.1016/j.jss.2022.111233>, <https://doi.org/10.1016/j.jss.2022.111233>
8. Sharma, Y., Arora, A.: A comprehensive review on permissions-based android malware detection: Y. sharma, a. arora. *International Journal of Information Security* **23**(3), 1877–1912 (2024)
9. Solanki, R.K., Laxmi, V., Gaur, M.S.: Evolution of android’s permission-based security model and challenges (2026), <https://arxiv.org/abs/2601.00252>
10. Taha, O., Karakra, A.: Tackling over privilege: Resolving access concerns in android mobile applications. In: *Proceedings of the 12th International Conference on Information Systems Security and Privacy - Volume 1: ICISSP*. pp. 136–143. INSTICC, SciTePress (2026). <https://doi.org/10.5220/0014243000004061>
11. TEBIB, M.E.A., André, P., Graa, M., Aktouf, O.E.K.: Handling over-privileged android applications based on the minimum permission set identification (2021)
12. Tuncay, G.S.: Android permissions: Evolution, attacks, and best practices. *IEEE Security & Privacy* **22**(6), 40–49 (2024). <https://doi.org/10.1109/MSEC.2024.3461629>
13. Wang, Y., Wang, Y., Wang, S., Liu, Y., Xu, C., Cheung, S.C., Yu, H., Zhu, Z.: Runtime permission issues in android apps: Taxonomy, practices, and ways forward. *IEEE Transactions on Software Engineering* **49**(1), 185–210 (2023). <https://doi.org/10.1109/TSE.2022.3148258>
14. Zhang, X., Yu, Z., Li, X., Zhang, C., Sun, C., Zhang, N., Deng, R.H.: Understanding the bad development practices of android custom permissions in the wild. *IEEE Transactions on Dependable and Secure Computing* **22**(4), 3208–3223 (2025). <https://doi.org/10.1109/TDSC.2024.3525049>